COMP 3311
Database Management Systems

Lab 3

SQL Basic DML Statement

Lab Topics

- The **SQL** basic **select** statement.
- Join operations.
- The **order by** clause.
- The **Oracle fetch** clause.
- The **collate** clause
- The dual table

To see the result of the example queries in these lab notes, run them in **Oracle SQL Developer** against the database created by the lab3db.sql script file.

Lab 3 Database

Student table

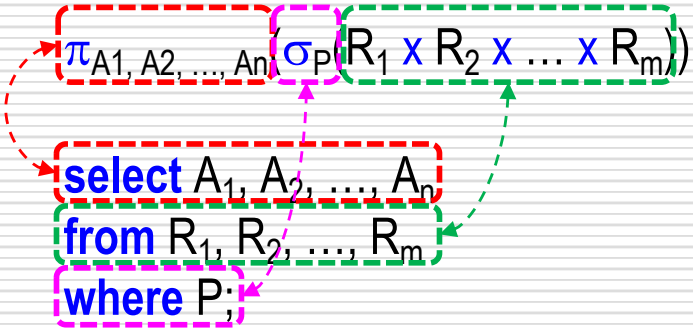
STUDENTID	FIRSTNAME	LASTNAME	EMAIL	PHONENO	ADMITYEAR	CGA	UNITID
13455789	Harry	Potter	cspotter	23581234	2023	2.76	COMP
15456789	Leo	Da Vinci	csdavinci	23585678	2023	2.72	COMP
13556789	Leo	Greenleaf	magreenleaf	23582468	2024	3.36	MATH
13456789	Ariana	Grande	csgrande	23581234	2024	2.82	COMP
15678989	Edith	Clarke	csclarke	23589876	2024	2.73	COMP
15678901	Albert	Einstein	cseinstein	23585678	2023	2.97	COMP
16789012	Robert	Redford	maredford	23582468	2024	2.57	MATH
14567890	David	Decker	eedecker	23589876	2024	1.49	ELEC
99987654	Lazzy	Lazy	cslazy	23581357	2024	null	COMP
26184624	Bruce	Wayne	eewayne	28261057	2023	2.47	ELEC
26184444	Donald	Trump	bstrump	28255057	2024	1.49	MGMT
26186666	Warren	Buffet	bsbuffet	28266027	2023	3.43	MGMT
28435018	David	Decker	bsdecker	27619435	2024	1.49	MGMT
66666666	Ferris	Bueller	bsbueller	28282727	2023	1.54	MGMT
15000655	Steve	Jobs	csjobs	26232244	2023	3.56	COMP
15085942	Bill	Gates	csgates	25678679	2024	3.4	COMP
28834512	Isaac	Newton	manewton	22861987	2023	2.98	MATH
28918856	Alan	Turing	maturing	26679834	2023	3.56	MATH
29873381	Nikola	Tesla	eetesla	25671983	2023	3.37	ELEC
13782973	Edith	Clarke	eeclarke	28340180	2023	3.37	ELEC
18792018	Elon	Musk	bsmusk	28659910	2024	2.76	MGMT
27182481	Buzzy	Bizy	bsbizy	26178891	2024	null	MGMT

AcademicUnit table

UNITID	UNITNAME	ROOMNO
COMP	Department of Computer Science and Engineering	3528
MATH	Department of Mathematics	3461
ELEC	Department of Electronic and Computer Engineering	2528
MGMT	Department of Management	4555
FINA	Department of Finance	4528
HUMA	Division of Humanities	1200

SELECT Statement

- Recall the correspondence between relational algebra and the **select** statement.



- Basic **select** statement syntax

select * { [**distinct**] *column* | *expression [alias], …*}
from *table, …*
where *condition;*

SELECT Statement — Examples

- Retrieve all table columns.

```
select *
from AcademicUnit;
```

UNITID	UNITNAME	ROOMNO
COMP	Department of Computer Science and Engineering	3528
MATH	Department of Mathematics	3461
ELEC	Department of Electronic and Computer Engineering	2528
MGMT	Department of Management	4555
FINA	Department of Finance	4528
HUMA	Division of Humanities	1200

- Retrieve specified table columns.

```
select unitId, unitName
from AcademicUnit;
```

UNITID	UNITNAME
COMP	Department of Computer Science and Engineering
MATH	Department of Mathematics
ELEC	Department of Electronic and Computer Engineering
MGMT	Department of Management
FINA	Department of Finance
HUMA	Division of Humanities

SELECT Statement — Removing Duplicate Results

- By default, the **select** statement returns all the qualifying records – including duplicate ones.

- The statement on the right returns all the unit ids from the Student table (22 values, one for each student).

- To remove duplicates, add the **distinct** keyword to the **select** clause:

```
select distinct unitId
from Student;
```

UNITID
MGMT
COMP
ELEC
MATH

```
select unitId
from Student;
```

UNITID
COMP
COMP
MATH
COMP
COMP
COMP
MATH
ELEC
COMP
ELEC
MGMT
MGMT
MGMT
MGMT
MGMT
COMP
COMP
MATH
MATH
ELEC
ELEC
MGMT
MGMT

SELECT Statement — Incorporating Arithmetic Operations

- Arithmetic operators like * , / , + , - can be included in a **select** statement.

```
select lastName, cga, cga+2.0
from Student;
```

LASTNAME	CGA	CGA+2
Potter	2.76	4.76
Da Vinci	2.72	4.72
Greenleaf	3.36	5.36
Grande	2.82	4.82
Clarke	2.73	4.73
Einstein	2.97	4.97
Redford	2.57	4.57
Decker	1.49	3.49
Lazy	null	null
Wayne	2.47	4.47
Trump	1.49	3.49
Buffet	3.43	5.43
Decker	1.49	3.49
Bueller	1.54	3.54
Jobs	3.56	5.56
Gates	3.4	5.4
Newton	2.98	4.98
Turing	3.56	5.56
Tesla	3.37	5.37
Clarke	3.37	5.37
Musk	2.76	4.76
Bizy	null	null

```
select lastName, cga, cga/2.0
from Student;
```

LASTNAME	CGA	CGA/2
Potter	2.76	1.38
Da Vinci	2.72	1.36
Greenleaf	3.36	1.68
Grande	2.82	1.41
Clarke	2.73	1.365
Einstein	2.97	1.485
Redford	2.57	1.285
Decker	1.49	0.745
Lazy	null	null
Wayne	2.47	1.235
Trump	1.49	0.745
Buffet	3.43	1.715
Decker	1.49	0.745
Bueller	1.54	0.77
Jobs	3.56	1.78
Gates	3.4	1.7
Newton	2.98	1.49
Turing	3.56	1.78
Tesla	3.37	1.685
Clarke	3.37	1.685
Musk	2.76	1.38
Bizy	null	null

Note: cga/2.0 will return the same result as cga/2 in SQL, this is different from some higher-level languages like C++.

SELECT Statement — Renaming A Query Result Column

- A column name in the query result can be renamed by using the **as** keyword.

```
select lastName as FamilyName
from Student;
```

FAMIYNAME
Potter
Da Vinci
Greenleaf
⋮

- The **select** statement can be used to output a column named Quarter CGA which displays the cga divided by 4.

```
select lastName, cga/4 as "Quarter CGA"
from Student;
```

LASTNAME	Quarter CGA
Potter	0.69
Da Vinci	0.68
Greenleaf	0.84
⋮	

Note: Double quotes are required around an alias only if it has an embedded space as in this example.

SELECT Statement — Concatenating Query Results

- The || operator can be used to concatenate two columns in a select statement.

```
select firstName || ' ' || lastName as "Full Name"
from Student;
```

Full Name
Harry Potter
Leo Da Vinci
Leo Greenleaf
⋮

Note: **Oracle** also provide the **concat** operator, but it takes only two arguments.

- The || operator can be used to add a string to the result.

```
select firstName || ' ' || lastName || ' studies in ' || unitId as "Description"
from Student;
```

Description
Harry Potter studies in COMP
Leo Da Vinci studies in COMP
Leo Greenleaf studies in MATH
⋮

Note: If double quotes are placed around a single word alias such as Description, then it is displayed as typed; otherwise, the alias name will be displayed in all capital letters.

SELECT Statement — Example Concatenation

- Concatenation can be used to make a query result more comprehensible.

```
select firstName ||' '|| lastName || '(' || studentId || ') ' || 'from the ' || unitId ||
  ' academic unit has CGA ' || CGA ||'.' ||' His/Her email is ' || email ||
  '@connect.ust.hk.' as lab3
from Student;
```

```
LAB3
Harry Potter(13455789) from the COMP academic unit has CGA 2.76. His/Her email is cspotter@connect.ust.hk.
Leo Da Vinci(15456789) from the COMP academic unit has CGA 2.72. His/Her email is csdavinci@connect.ust.hk.
Leo Greenleaf(13556789) from the MATH academic unit has CGA 3.36. His/Her email is magreenleaf@connect.ust.hk.
Ariana Grande(13456789) from the COMP academic unit has CGA 2.82. His/Her email is csgrande@connect.ust.hk.
Edith Clarke(15678989) from the COMP academic unit has CGA 2.73. His/Her email is csclarke@connect.ust.hk.
Albert Einstein(15678901) from the COMP academic unit has CGA 2.97. His/Her email is cseinstein@connect.ust.hk.
Robert Redford(16789012) from the MATH academic unit has CGA 2.57. His/Her email is maredford@connect.ust.hk.
David Decker(14567890) from the ELEC academic unit has CGA 1.49. His/Her email is eedecker@connect.ust.hk.
Lazzy Lazy(99987654) from the COMP academic unit has CGA . His/Her email is cslazy@connect.ust.hk.
Bruce Wayne(26184624) from the ELEC academic unit has CGA 2.47. His/Her email is eewayne@connect.ust.hk.
Donald Trump(26184444) from the MGMT academic unit has CGA 1.49. His/Her email is bstrump@connect.ust.hk.
Warren Buffet(26186666) from the MGMT academic unit has CGA 3.43. His/Her email is bsbuffet@connect.ust.hk.
David Decker(28435018) from the MGMT academic unit has CGA 1.49. His/Her email is bsdecker@connect.ust.hk.
Ferris Bueller(66666666) from the MGMT academic unit has CGA 1.54. His/Her email is bsbueller@connect.ust.hk.
Steve Jobs(15000655) from the COMP academic unit has CGA 3.56. His/Her email is csjobs@connect.ust.hk.
Bill Gates(15085942) from the COMP academic unit has CGA 3.4. His/Her email is csgates@connect.ust.hk.
⋮
```

SELECT Statement — WHERE Clause

- The **where** clause restricts the **select** statement so that only specified rows from a table are retrieved.

```
select *
from AcademicUnit
where unitId='COMP';
```

UNITID	UNITNAME	ROOMNO
COMP	Department of Computer Science and Engineering	3528

The string 'COMP' in the **where** clause is case sensitive.

- **WHERE** clause comparison operators:

=	equal	>	greater than	<	less than
>=	greater or equal	<=	less or equal	<> *or* !=	not equal

Examples:

```
select *
from Student
where cga<>2.5;
```

```
select *
from Student
where cga<=1.9;
```

WHERE Clause — Boolean Operators (1)

- **and**

```
select firstName, lastName, cga, unitId
from Student
where cga>=2 and unitId='ELEC';
```

FIRSTNAME	LASTNAME	CGA	UNITID
Bruce	Wayne	2.47	ELEC
Nikola	Tesla	3.37	ELEC
Edith	Clarke	3.37	ELEC

- **not**

```
select firstName, lastName, cga, unitId
from Student
where not (unitId='COMP' or unitId='MGMT');
```

FIRSTNAME	LASTNAME	CGA	UNITID
Leo	Greenleaf	3.36	MATH
Robert	Redford	2.57	MATH
David	Decker	1.49	ELEC
Bruce	Wayne	2.47	ELEC
Isaac	Newton	2.98	MATH
Alan	Turing	3.56	MATH
Nikola	Tesla	3.37	ELEC
Edith	Clarke	3.37	ELEC

WHERE Clause — Boolean Operators (2)

- **or**

Query: Find students whose cga is greater than or equal to 3 or who are in the ELEC unit regardless of their cga value.

```
select firstName, lastName, cga, unitId
from Student
where cga>=3 or unitId='ELEC';
```

FIRSTNAME	LASTNAME	CGA	UNITID
Leo	Greenleaf	3.36	MATH
David	Decker	1.49	ELEC
Bruce	Wayne	2.47	ELEC
Warren	Buffet	3.43	MGMT
Steve	Jobs	3.56	COMP
Bill	Gates	3.4	COMP
Alan	Turing	3.56	MATH
Nikola	Tesla	3.37	ELEC
Edith	Clarke	3.37	ELEC

WHERE Clause — Boolean Operator Precedence (1)

- The **and** operator has higher precedence than the **or** operator.

Query: Find students from the COMP unit *plus* also students from the MATH unit with cga>=3.

```
select firstName, lastName, cga, unitId
from Student
where unitId='COMP' or unitId='MATH' and cga>=3;
```

FIRSTNAME	LASTNAME	CGA	UNITID
Harry	Potter	2.76	COMP
Leo	Da Vinci	2.72	COMP
Leo	Greenleaf	3.36	MATH
Ariana	Grande	2.82	COMP
Edith	Clarke	2.73	COMP
Albert	Einstein	2.97	COMP
Lazzy	Lazy	null	COMP
Steve	Jobs	3.56	COMP
Bill	Gates	3.4	COMP
Alan	Turing	3.56	MATH

WHERE Clause — Boolean Operator Precedence (2)

- Use parenthesis to change the precedence order.

Query: Find students with cga>=3, from either the COMP or the MATH academic units.

```
select firstName, lastName, cga, unitId
from Student
where (unitId='COMP' or unitId='MATH') and cga>=3;
```

FIRSTNAME	LASTNAME	CGA	UNITID
Leo	Greenleaf	3.36	MATH
Steve	Jobs	3.56	COMP
Bill	Gates	3.4	COMP
Alan	Turing	3.56	MATH

WHERE Clause — Condition Operators

- Range of values: **between** / **not between**

```
select firstName, lastName, cga, unitId
from Student
where cga between 2.8 and 3;
```

FIRSTNAME	LASTNAME	CGA	UNITID
Ariana	Grande	2.82	COMP
Albert	Einstein	2.97	COMP
Isaac	Newton	2.98	MATH

- Set membership: **in** / **not in**

```
select firstName, lastName, cga, unitId
from Student
where unitId in ('ELEC', 'MATH');
```

FIRSTNAME	LASTNAME	CGA	UNITID
Leo	Greenleaf	3.36	MATH
Robert	Redford	2.57	MATH
David	Decker	1.49	ELEC
Bruce	Wayne	2.47	ELEC
Isaac	Newton	2.98	MATH
Alan	Turing	3.56	MATH
Nikola	Tesla	3.37	ELEC
Edith	Clarke	3.37	ELEC

WHERE Clause — String Matching (1)

- **like** (for matching patterns)

  % can match zero or more characters.

  Query: Find students whose first name contains a "a" anywhere in the name.

```
select *
from Student
where firstName like '%a%';
```

STUDENTID	FIRSTNAME	LASTNAME	EMAIL	PHONENO	ADMITYEAR	CGA	UNITID
13455789	Harry	Potter	cspotter	23581234	2023	2.76	COMP
13456789	Ariana	Grande	csgrande	23581234	2024	2.82	COMP
14567890	David	Decker	eedecker	23589876	2024	1.49	ELEC
99987654	Lazzy	Lazy	cslazy	23581357	2024	null	COMP
26184444	Donald	Trump	bstrump	28255057	2024	1.49	MGMT
26186666	Warren	Buffet	bsbuffet	28266027	2023	3.43	MGMT
28435018	David	Decker	bsdecker	27619435	2024	1.49	MGMT
28834512	Isaac	Newton	manewton	22861987	2023	2.98	MATH
28918856	Alan	Turing	maturing	26679834	2023	3.56	MATH
29873381	Nikola	Tesla	eetesla	25671983	2023	3.37	ELEC

WHERE Clause — String Matching (2)

- **like** (for matching patterns)

  _ (underscore) matches exactly one character.

  Query: Find students whose first name contains a "u" as the second character.

```
select *
from Student
where firstName like '_u%';
```

STUDENTID	FIRSTNAME	LASTNAME	EMAIL	PHONENO	ADMITYEAR	CGA	UNITID
27182481	Buzzy	Bizy	bsbizy	26178891	2024	null	MGMT

WHERE Clause — String Matching (3)

- **regexp_like** function (for matching regular expressions)

  Syntax: **regexp_like**(*attribute-name*, *regular-expression*, *match-parameter*)

  match-parameter: i → case insensitive; c → case sensitive.

Query: Find students with a double vowel in their last name.

```
select *
from Student
where regexp_like(lastName, '([aeiou])\1', 'i');
```

STUDENTID	FIRSTNAME	LASTNAME	EMAIL	PHONENO	ADMITYEAR	CGA	UNITID
13556789	Leo	Greenleaf	magreenleaf	23582468	2024	3.36	MATH

For information on the regular expressions supported by Oracle see https://docs.oracle.com/cd/B12037_01/server.101/b10759/ap_posix001.htm#i690819.

For examples of regular expression use in Oracle see https://www.salvis.com/blog/2018/09/28/regular-expressions-sql-examples/.

For testing your regular expressions see https://www.regextester.com/ (use the PCRE option).

Null Value Handling

- Null value in the **where** clause: **is null** / **is not null**

```
select firstName, lastName, cga, unitId
from Student
where cga is null;
```

FIRSTNAME	LASTNAME	CGA	UNITID
Lazzy	Lazy	null	COMP
Buzzy	Bizy	null	MGMT

Note: Cannot use **where** cga=**null**. This will return no result as null values are treated in a special way.

- Null value replacement: **coalesce** function

```
select firstName, lastName, coalesce(cga, 0) as cga, unitId
from Student
where cga is null;
```

FIRSTNAME	LASTNAME	CGA	UNITID
Lazzy	Lazy	0	COMP
Buzzy	Bizy	0	MGMT

Note: The replacement value must be of a compatible type with the value being replaced. In this example it must be of the same types as cga.

Cartesian Product

- Cartesian product combines each row of one table with every row of another table.

```
select firstName, lastName, unitName
from Student, AcademicUnit;
```

Note: If the Student table has 22 entries and the AcademicUnit table has 6 entries, then the query result has (22x6) 132 entries.

FIRSTNAME	LASTNAME	UNITNAME
Harry	Potter	Department of Computer Science and Engineering
Leo	Da Vinci	Department of Computer Science and Engineering
Leo	Greenleaf	Department of Computer Science and Engineering
Ariana	Grande	Department of Computer Science and Engineering
Edith	Clarke	Department of Computer Science and Engineering
Albert	Einstein	Department of Computer Science and Engineering
Robert	Redford	Department of Computer Science and Engineering
David	Decker	Department of Computer Science and Engineering
Lazzy	Lazy	Department of Computer Science and Engineering
Bruce	Wayne	Department of Computer Science and Engineering
Donald	Trump	Department of Computer Science and Engineering
Warren	Buffet	Department of Computer Science and Engineering
David	Decker	Department of Computer Science and Engineering
Ferris	Bueller	Department of Computer Science and Engineering
Steve	Jobs	Department of Computer Science and Engineering
Bill	Gates	Department of Computer Science and Engineering
Isaac	Newton	Department of Computer Science and Engineering
Alan	Turing	Department of Computer Science and Engineering
Nikola	Tesla	Department of Computer Science and Engineering
Edith	Clarke	Department of Computer Science and Engineering
⋮		

Harry	Potter	Division of Humanities
Leo	Da Vinci	Division of Humanities
Leo	Greenleaf	Division of Humanities
Ariana	Grande	Division of Humanities
Edith	Clarke	Division of Humanities
Albert	Einstein	Division of Humanities
Robert	Redford	Division of Humanities
David	Decker	Division of Humanities
Lazzy	Lazy	Division of Humanities
Bruce	Wayne	Division of Humanities
Donald	Trump	Division of Humanities
Warren	Buffet	Division of Humanities
David	Decker	Division of Humanities
Ferris	Bueller	Division of Humanities
Steve	Jobs	Division of Humanities
Bill	Gates	Division of Humanities
Isaac	Newton	Division of Humanities
Alan	Turing	Division of Humanities
Nikola	Tesla	Division of Humanities
Edith	Clarke	Division of Humanities
Elon	Musk	Division of Humanities
Buzzy	Bizy	Division of Humanities

Join (1)

- Join is a Cartesian product followed by a selection.

```
select firstName, lastName, unitName
from Student, AcademicUnit
where Student.unitId=AcademicUnit.unitId;
```

FIRSTNAME	LASTNAME	UNITNAME
Harry	Potter	Department of Computer Science and Engineering
Leo	Da Vinci	Department of Computer Science and Engineering
Leo	Greenleaf	Department of Mathematics
Ariana	Grande	Department of Computer Science and Engineering
Edith	Clarke	Department of Computer Science and Engineering
Albert	Einstein	Department of Computer Science and Engineering
Robert	Redford	Department of Mathematics
David	Decker	Department of Electronic and Computer Engineering
Lazzy	Lazy	Department of Computer Science and Engineering
Bruce	Wayne	Department of Electronic and Computer Engineering
Donald	Trump	Department of Management
Warren	Buffet	Department of Management
David	Decker	Department of Management
Ferris	Bueller	Department of Management
Steve	Jobs	Department of Computer Science and Engineering
Bill	Gates	Department of Computer Science and Engineering
Isaac	Newton	Department of Mathematics
Alan	Turing	Department of Mathematics
Nikola	Tesla	Department of Electronic and Computer Engineering
Edith	Clarke	Department of Electronic and Computer Engineering
Elon	Musk	Department of Management
Buzzy	Bizy	Department of Management

Note: Attributes names need to be qualified with table names if they are ambiguous. For example, unitId is an attribute of both the Student and AcademicUnit tables in this example.

If the Student table has 22 entries and the AcademicUnit table has 6 entries, then the query result has 22 entries, *one for each student*.

Student(studentId, firstName, lastName, email, phoneNo , admitYear, cga, unitId)
AcademicUnit(unitId, unitName, roomNo)

Join (2)

- A join can also be specified as follows.

```
select firstName, lastName, unitName
from Student join AcademicUnit on Student.unitId=AcademicUnit.unitId;
```

FIRSTNAME	LASTNAME	UNITNAME
Harry	Potter	Department of Computer Science and Engineering
Leo	Da Vinci	Department of Computer Science and Engineering
Leo	Greenleaf	Department of Mathematics
Ariana	Grande	Department of Computer Science and Engineering
Edith	Clarke	Department of Computer Science and Engineering
Albert	Einstein	Department of Computer Science and Engineering
Robert	Redford	Department of Mathematics
David	Decker	Department of Electronic and Computer Engineering
Lazzy	Lazy	Department of Computer Science and Engineering
Bruce	Wayne	Department of Electronic and Computer Engineering
Donald	Trump	Department of Management
Warren	Buffet	Department of Management
David	Decker	Department of Management
Ferris	Bueller	Department of Management
Steve	Jobs	Department of Computer Science and Engineering
Bill	Gates	Department of Computer Science and Engineering
Isaac	Newton	Department of Mathematics
Alan	Turing	Department of Mathematics
Nikola	Tesla	Department of Electronic and Computer Engineering
Edith	Clarke	Department of Electronic and Computer Engineering
Elon	Musk	Department of Management
Buzzy	Bizy	Department of Management

Student(studentId, firstName, lastName, email, phoneNo , admitYear, cga, unitId)
AcademicUnit(unitId, unitName, roomNo)

Join With Conditions

- Additional conditions in the **where** clause along with a join condition further restricts the tuples selected.

```
select *
from Student, AcademicUnit
where Student.unitId=AcademicUnit.unitId
        and Student.unitId='COMP'
        and cga>2.5;
```

STUDENTID	FIRSTNAME	LASTNAME	EMAIL	PHONENO	ADMITYEAR	CGA	UNITID	UNITID_1	UNITNAME	ROOMNO
13455789	Harry	Potter	cspotter	23581234	2023	2.76	COMP	COMP	Department of Computer Science and Engineering	3528
15456789	Leo	Da Vinci	csdavinci	23585678	2023	2.72	COMP	COMP	Department of Computer Science and Engineering	3528
13456789	Ariana	Grande	csgrande	23581234	2024	2.82	COMP	COMP	Department of Computer Science and Engineering	3528
15678989	Edith	Clarke	csclarke	23589876	2.73	2024	COMP	COMP	Department of Computer Science and Engineering	3528
15678901	Albert	Einstein	cseinstein	23585678	2.97	2023	COMP	COMP	Department of Computer Science and Engineering	3528
15000655	Steve	Jobs	csjobs	26232244	3.56	2023	COMP	COMP	Department of Computer Science and Engineering	3528
15085942	Bill	Gates	csgates	25678679	3.4	2024	COMP	COMP	Department of Computer Science and Engineering	3528

Note: The join attribute, unitId, repeats in the query result.

Natural Join

- A natural join combines the rows of the tables if columns with identical names match on their values and keeps only one of the join columns.
- For the tables Student and AcademicUnit, only rows with identical values in the column unitId will be merged, so students with unitId ='COMP' will merge with the academic unit with unitId = 'COMP'.

```
select *
from Student natural join AcademicUnit
where unitId='COMP'
        and cga>2.5;
```

Note: The join (common) attribute, unitId, *does not* repeat in the query result.

UNITID	STUDENTID	FIRSTNAME	LASTNAME	EMAIL	PHONENO	ADMITYEAR	CGA	UNITNAME	ROOMNO
COMP	13455789	Harry	Potter	cspotter	23581234	2023	2.76	Department of Computer Science and Engineering	3528
COMP	15456789	Leo	Da Vinci	csdavinci	23585678	2023	2.72	Department of Computer Science and Engineering	3528
COMP	13456789	Ariana	Grande	csgrande	23581234	2024	2.82	Department of Computer Science and Engineering	3528
COMP	15678989	Edith	Clarke	csclarke	23589876	2024	2.73	Department of Computer Science and Engineering	3528
COMP	15678901	Albert	Einstein	cseinstein	23585678	2023	2.97	Department of Computer Science and Engineering	3528
COMP	15000655	Steve	Jobs	csjobs	26232244	2023	3.56	Department of Computer Science and Engineering	3528
COMP	15085942	Bill	Gates	csgates	25678679	2024	3.4	Department of Computer Science and Engineering	3528

Student(studentId, firstName, lastName, email, phoneNo , admitYear, cga, unitId)
AcademicUnit(unitId, unitName, roomNo)

ORDER BY Clause (1)

- **asc** ascending order (default)

```
select studentId, firstName, lastName, cga
from Student
where unitId='COMP'
order by cga;
```

STUDENTID	FIRSTNAME	LASTNAME	CGA
15456789	Leo	Da Vinci	2.72
15678989	Edith	Clarke	2.73
13455789	Harry	Potter	2.76
13456789	Ariana	Grande	2.82
15678901	Albert	Einstein	2.97
15085942	Bill	Gates	3.4
15000655	Steve	Jobs	3.56
99987654	Lazzy	Lazy	null

- **desc** descending order

```
select studentId, firstName, lastName, cga
from Student
where unitId='COMP'
order by cga desc;
```

STUDENTID	FIRSTNAME	LASTNAME	CGA
99987654	Lazzy	Lazy	null
15000655	Steve	Jobs	3.56
15085942	Bill	Gates	3.4
15678901	Albert	Einstein	2.97
13456789	Ariana	Grande	2.82
13455789	Harry	Potter	2.76
15678989	Edith	Clarke	2.73
15456789	Leo	Da Vinci	2.72

Note: In **SQL**, a null value is always the largest value in sort order. Thus, null values appear *after* all other values in ascending sort order and *before* all values in descending sort order.

ORDER BY Clause (2)

- Sort on multiple columns

```
select firstName, lastName, cga, unitId
from Student
order by unitId asc, lastName desc;
```

FIRSTNAME	LASTNAME	CGA	UNITID
Harry	Potter	2.76	COMP
Lazzy	Lazy	null	COMP
Steve	Jobs	3.56	COMP
Ariana	Grande	2.82	COMP
Bill	Gates	3.4	COMP
Albert	Einstein	2.97	COMP
Leo	Da Vinci	2.72	COMP
Edith	Clarke	2.73	COMP
Bruce	Wayne	2.47	ELEC
Nikola	Tesla	3.37	ELEC
David	Decker	1.49	ELEC
Edith	Clarke	3.37	ELEC
Alan	Turing	3.56	MATH
Robert	Redford	2.57	MATH
Isaac	Newton	2.98	MATH
Leo	Greenleaf	3.36	MATH
Donald	Trump	1.49	MGMT
Elon	Musk	2.76	MGMT
David	Decker	1.49	MGMT
Warren	Buffet	3.43	MGMT
Ferris	Bueller	1.54	MGMT
Buzzy	Bizy	null	MGMT

Oracle FETCH Clause (1)

- The fetch clause limits retrieval to only a portion of the rows generated by an SQL query*.

[offset offset rows]
fetch [first | next] [row_count | percent percent] [row | rows] [only | with ties]

  - offset specifies the number of rows to skip before the retrieval of rows starts; it is optional.
  - row_count specifies the number of rows to retrieve.
  - percent percent specifies the percentage of rows to retrieve.
  - only returns exactly the number of rows or percentage of rows specified.
  - with ties returns additional rows that have the same sort key value as the last row fetched (requires an order by clause in the query).

* The syntax of the fetch clause is Oracle Database specific.

Oracle FETCH Clause (2)

Examples

Query: Find any three the students who have the highest cga.

```
select firstName, lastName, cga, unitId
from Student
where cga is not null
order by cga desc
fetch first 3 rows only;
```

FIRSTNAME	LASTNAME	CGA	UNITID
Steve	Jobs	3.56	COMP
Alan	Turing	3.56	MATH
Warren	Buffet	3.43	MGMT

Query: Find all students who have the third highest cga.

```
select firstName, lastName, cga, unitId
from Student
where cga is not null
order by cga desc
offset 2 rows
fetch next 1 row with ties;
```

FIRSTNAME	LASTNAME	CGA	UNITID
Warren	Buffet	3.43	MGMT

Note: Since null values always sorts as the largest value, the where clause is needed to exclude the student Lazzy Lazy who has a null cga.

SQL CASE statement

- The **case** statement evaluates the predicates specified in the **when** clauses and returns the value in the **then** clause of the first predicate that evaluates to true.
  - If no predicates evaluate to true, it returns the value in the **else** clause.
  - If there is no **else** clause and no predicates are true, it returns null.

FIRSTNAME	LASTNAME	CGA	UNITID
Harry	Potter	2.76	COMP
Leo	Da Vinci	2.72	COMP
Leo	Greenleaf	3.36	MATH
Ariana	Grande	2.82	COMP
Edith	Clarke	2.73	COMP
Albert	Einstein	2.97	COMP
Robert	Redford	2.57	MATH
David	Decker	1.49	ELEC
Lazzy	Lazy	0	COMP
Bruce	Wayne	2.47	ELEC
Donald	Trump	1.49	MGMT
Warren	Buffet	3.43	MGMT
David	Decker	1.49	MGMT
Ferris	Bueller	1.54	MGMT
Steve	Jobs	3.56	COMP
Bill	Gates	3.4	COMP
Isaac	Newton	2.98	MATH
Alan	Turing	3.56	MATH
Nikola	Tesla	3.37	ELEC
Edith	Clarke	3.37	ELEC
Elon	Musk	2.76	MGMT
Buzzy	Bizy	0	MGMT

Query: Find the first name, last name, cga and unit id of all students replacing a null cga by 0.

```
select firstName, lastName, case when cga is null then 0 else cga end as
        cga, unitId
from Student;
```

Note: The replacement value must be of a compatible type with the value being replaced.

COLLATE Clause (1)

- Collation determines how strings are compared, which has a direct impact on ordering and equality tests between strings.
- Oracle Database allows specification of the collation used for columns that hold string data, allowing case insensitive queries to be performed, as well as control of the output order of queried data.
- Collation can be set at the column-, table-, schema-, session-, database- or statement-level.
- Here we will only show how to set collation at the statement-level.

COLLATE Clause (2)

- There are two basic types of collation.
  - binary: Ordering and comparisons of string data are based on the numeric value of the characters in the strings.
  - linguistic: Ordering and comparisons of string data are based on the alphabetic sequence of the characters, regardless of their numeric values.
- When using collations there are two suffixes for binary collation that alter the behaviour of sorts and comparisons.
  - ci Case insensitive but accent sensitive.
  - ai Both case and accent insensitive.
- The default binary collation is both case and accent sensitive if no collation extension is specified.

COLLATE Clause (3)

- Suppose the student records shown on the right are in the database.

FIRSTNAME	LASTNAME	CGA	UNITID
Löwen	Bräun	2.76	MGMT
LÖwen	BrÄun	2.72	COMP
Lowen	Braun	3.36	ELEC
LOwen	BrAun	2.82	MATH

Example: collation in **order by** clause.

- **collate** binary (the default; can omit)

```
select firstName, lastName, cga, unitId
from Student
order by lastName collate binary;
```

FIRSTNAME	LASTNAME	CGA	UNITID
LOwen	BrAun	2.82	MATH
Lowen	Braun	3.36	ELEC
LÖwen	BrÄun	2.72	COMP
Löwen	Bräun	2.76	MGMT

- **collate** binary_ci

```
select firstName, lastName, cga, unitId
from Student
order by lastName collate binary_ci;
```

FIRSTNAME	LASTNAME	CGA	UNITID
Lowen	Braun	3.36	ELEC
LOwen	BrAun	2.82	MATH
Löwen	Bräun	2.76	MGMT
LÖwen	BrÄun	2.72	COMP

- **collate** binary_ai

```
select firstName, lastName, cga, unitId
from Student
order by lastName collate binary_ai;
```

FIRSTNAME	LASTNAME	CGA	UNITID
Löwen	Bräun	2.76	MGMT
LOwen	BrAun	2.82	MATH
Lowen	Braun	3.36	ELEC
LÖwen	BrÄun	2.72	COMP

COLLATE Clause (4)

Example: collation in **where** clause.

Query: Find students whose last name is Braun (no **collate** clause).

```
select firstName, lastName, cga, unitId
from Student
where lastName='Braun';
```

FIRSTNAME	LASTNAME	CGA	UNITID
Löwen	Bräun	2.76	MGMT
LÖwen	BrÄun	2.72	COMP
Lowen	Braun	3.36	ELEC
LOwen	BrAun	2.82	MATH

FIRSTNAME	LASTNAME	CGA	UNITID
Lowen	Braun	3.36	ELEC

Query: Find students whose last name is Braun (with **collate** binary_ci).

```
select firstName, lastName, cga, unitId
from Student
where lastName='Braun' collate binary_ci;
```

FIRSTNAME	LASTNAME	CGA	UNITID
Lowen	Braun	3.36	ELEC
LOwen	BrAun	2.82	MATH

Query: Find students whose last name is Braun (with **collate** binary_ai)

```
select firstName, lastName, cga, unitId
from Student
where lastName='Braun' collate binary_ai;
```

FIRSTNAME	LASTNAME	CGA	UNITID
Löwen	Bräun	2.76	MGMT
LÖwen	BrÄun	2.72	COMP
Lowen	Braun	3.36	ELEC
LOwen	BrAun	2.82	MATH

The dual Table

- The dual table is an Oracle built-in table for SQL queries that do not logically have table names.

```
select 'The results of the queries are: '
from dual;
```

will output the string: The results of the queries are:

Note: To suppress the output of table column headers in the Script Output pane of **Oracle SQL Developer**, place the **SQL*Plus** command "set heading off" in a script file *before* the **SQL** statement(s) whose result column headers you want to suppress. Use the command "set heading on" to again show the column headers for the result of **SQL** statements.